HƯỚNG DẪN CHI TIẾT TRIỂN KHAI
HỆ THỐNG TUTORRAG CHO NGƯỜI TỰ HỌC

Từ tài liệu học tập thô đến hệ thống Grounded Reasoning RAG hoàn chỉnh

Mục tiêu tài liệu
Tài liệu này mô tả **từng bước, từng đầu vào, từng đầu ra** để xây dựng một hệ thống RAG có khả năng **truy hồi, tổng hợp và suy luận dựa trên bằng chứng** từ tài liệu học tập cá nhân của người tự học. Trọng tâm là làm rõ:
- Bước nào cần dữ liệu gì.
- Bước đó xử lý chính xác điều gì.
- Output có dạng gì.
- Output ấy được đưa sang bước tiếp theo như thế nào.
- Những module nào nên xuất hiện trong codebase.

Phiên bản: 1.0
Định dạng biên dịch khuyến nghị: `XeLaTeX`

Mục lục

1 Tổng quan bài toán **3**

2 Bức tranh toàn bộ pipeline **3**

3 Chuẩn đầu vào và đầu ra cần thống nhất từ đầu **5**

3.1 Đầu vào gốc của toàn hệ thống 5

3.2 Đầu ra cuối cùng của toàn hệ thống 5

4 Giai đoạn A – Khởi tạo bài toán và thiết kế nền tảng **5**

4.1 Cấu trúc dự án dự kiến trên GitHub 9

5 Giai đoạn B – Xây dựng knowledge base từ tài liệu gốc **18**

6 Giai đoạn C – Làm knowledge base thông minh hơn vector search thuần túy **23**

7 Giai đoạn D – Online pipeline cho câu hỏi có suy luận **26**

8 Giai đoạn E – Orchestration, API, đánh giá và triển khai **32**

9 Mối liên hệ input-output giữa các bước **34**

9.1 Chuỗi offline 34

9.2 Chuỗi online 34

10 Kế hoạch triển khai đề xuất theo tuần **35**

11 Definition of Done cho từng lớp hệ thống **35**

11.1 Offline ingestion hoàn thành khi nào? 35

11.2 Retrieval hoàn thành khi nào? 35

11.3 Reasoning pipeline hoàn thành khi nào? 36

12 Những lỗi kiến trúc thường gặp cần tránh **36**

13 Điều chỉnh kỹ thuật mở rộng theo góp ý triển khai **36**

13.1 Điều chỉnh cho Bước 5 và Bước 6 – Parsing và Normalize 36

13.2 Điều chỉnh cho Bước 11 và Bước 12 – Concept index, relation graph và hướng GraphRAG 37
13.3 Điều chỉnh cho Bước 15, Bước 16 và Bước 17 – Agentic AI và ReAct planner 38
13.4 Điều chỉnh cho Bước 20 và Bước 23 – Verification, Self-RAG và Ragas 38
13.5 Các thách thức đặc thù với dữ liệu Bách Khoa và tiếng Việt 39

14 Phụ lục A – Data contracts khuyến nghị 40
14.1 QueryAnalysis 40
14.2 EvidenceItem 40
14.3 FinalAnswer 40

15 Phụ lục B – Skeleton codebase nên có 41

16 Kết luận 42

1 Tổng quan bài toán

Bạn không xây một chatbot chỉ biết nhắc lại tài liệu. Bạn đang xây một **trợ lý tự học có suy luận dựa trên tài liệu cá nhân**. Điều này có nghĩa là hệ thống phải làm được bốn việc đồng thời:

1. Lưu trữ và quản lý tài liệu học tập theo môn, chương, loại tài liệu.
2. Truy hồi đúng những đoạn liên quan đến câu hỏi hiện tại.
3. Kết nối nhiều đoạn làm bằng chứng để suy luận, so sánh, giải thích *why/how*.
4. Trả lời có trích nguồn, có kiểm tra lại độ bám sát evidence, không nói tự do ngoài tài liệu.

Nguyên tắc thiết kế cốt lõi
Grounded first, reasoning second. Nghĩa là hệ thống chỉ được suy luận *trên* bằng chứng đã truy hồi, không được tưởng tượng thêm tri thức nếu tài liệu không hỗ trợ.
Pipeline hai miền. Một miền là **offline ingestion/indexing**; miền còn lại là **online query/reasoning**.
Mọi bước đều có contract dữ liệu rõ ràng. Không truyền dữ liệu mơ hồ giữa các module.

2 Bức tranh toàn bộ pipeline

Bước	Tên bước	Input chính	Output chính
1	Xác định phạm vi sản phẩm	Mục tiêu đề tài, user persona, loại môn học	Bộ yêu cầu chức năng, use case, non-functional requirements
2	Thiết kế mô hình dữ liệu	Use case bước 1	Schema cho document, chunk, query, answer, feedback
3	Thiết kế cấu trúc thư mục và hạ tầng	Schema + chọn stack	Project skeleton, storage layout, service map
4	Upload và lưu file gốc	PDF/DOCX/PPTX/TXT từ người dùng	File gốc + metadata document ban đầu
5	Parse tài liệu	File gốc + metadata document	Văn bản thô theo trang/khối/heading

Bước	Tên bước	Input chính	Output chính
6	Normalize nội dung	Văn bản parse được	Nội dung sạch, bỏ nhiễu, giữ cấu trúc học thuật
7	Chunking theo cấu trúc	Nội dung sạch	Danh sách chunk có ranh giới hợp lý
8	Gắn metadata chunk	Chunks + document metadata	Chunk records đầy đủ metadata
9	Tạo vector dense	Chunk records	Dense embeddings
10	Tạo chỉ mục sparse	Chunk records	Sparse vectors / BM25 features
11	Xây concept index	Chunk records	Tập khái niệm, định nghĩa, từ khóa học thuật
12	Xây relation graph	Concept index + chunks	Đồ thị quan hệ khái niệm / prerequisite / minh họa
13	Upsert vào vector DB	Dense + sparse + metadata	Collection truy hồi hoàn chỉnh
14	Nhận query online	User question + session context	Query request chuẩn hóa
15	Query understanding	Query request	Query type, intent, subject filter, độ khó
16	Query decomposition	Query type + query gốc	Danh sách sub-question / sub-query
17	Hybrid retrieval nhiều vòng	Sub-query + indexes	Danh sách candidate evidence
18	Merge, dedupe, rerank	Candidate evidence	Evidence set xếp hạng cao
19	Reasoning synthesis	Evidence set + question	Câu trả lời sơ bộ + claims + citations
20	Verification	Draft answer + evidence set	Final answer đã kiểm tra support
21	Logging và feedback	Final answer + hành vi user	Bản ghi đánh giá phiên hỏi đáp
22	API và orchestration	Toàn bộ modules ở trên	Dịch vụ backend chạy được
23	Evaluation	Bộ câu hỏi chuẩn + logs	Báo cáo chất lượng retrieval/reasoning
24	Deployment	Ứng dụng hoàn chỉnh	Môi trường triển khai local/staging/production

3 Chuẩn đầu vào và đầu ra cần thống nhất từ đầu

3.1 Đầu vào gốc của toàn hệ thống

- Tài liệu học tập: PDF, DOCX, PPTX, Markdown, TXT.
- Metadata do người dùng hoặc hệ thống cung cấp: môn học, tên tài liệu, loại tài liệu, ngôn ngữ.
- Câu hỏi của user: một truy vấn ngôn ngữ tự nhiên, có thể kèm theo ngữ cảnh phiên học.

3.2 Đầu ra cuối cùng của toàn hệ thống

- Một câu trả lời dễ hiểu, có suy luận dựa trên evidence.
- Danh sách trích dẫn nguồn: tài liệu, trang, chunk.
- Chỉ số tin cậy nội bộ và logs để đánh giá sau này.

4 Giai đoạn A – Khởi tạo bài toán và thiết kế nền tảng

Bước 1. Xác định phạm vi sản phẩm và kịch bản sử dụng

Mục tiêu của bước: Biến ý tưởng mơ hồ "làm RAG cho người tự học" thành một bài toán sản phẩm cụ thể, có tiêu chí hoàn thành rõ ràng.
Input: Mô tả đề tài, đối tượng người dùng, loại tài liệu mà user có, dạng câu hỏi mà hệ thống cần trả lời.
Xử lý chính: Xác định user persona, mô tả 5–10 use case cốt lõi, chốt rõ hệ thống có nhiệm vụ **truy hồi + suy luận + trích nguồn** thay vì chỉ tóm tắt.
Output: Tài liệu yêu cầu gồm: danh sách chức năng, giới hạn hệ thống, non-functional requirements như độ trễ, khả năng mở rộng, khả năng giải thích.
Output này trở thành input cho: Bước 2 – Thiết kế schema dữ liệu.
Module / file nên có: `docs/problem_statement.md`, `docs/use_cases.md`, `docs/requirements.md`

Các việc phải làm cụ thể

1. Xác định rõ hệ thống phục vụ ai: sinh viên đại học, học sinh, người tự

học nhiều môn, người ôn thi.

2. Viết danh sách use case. Ví dụ:
   - Hỏi định nghĩa một khái niệm.
   - So sánh hai khái niệm giữa hai chương.
   - Hỏi tại sao lời giải trong tài liệu lại chọn phương pháp A thay vì B.
   - Yêu cầu tóm tắt một chương theo mức dễ hiểu.
   - Sinh câu hỏi ôn tập từ tài liệu đã học.
3. Xác định những gì hệ thống **không** làm ở phiên bản đầu. Ví dụ: chưa chấm bài tự luận dài, chưa truy cập web ngoài tài liệu cá nhân.
4. Chốt các tiêu chí chất lượng:
   - Trích dẫn phải chỉ ra đúng tài liệu và trang.
   - Khi evidence không đủ, hệ thống phải nói "chưa đủ bằng chứng".
   - Truy hồi phải lọc theo đúng user và đúng môn học.

Mẫu output nên có

```
{
  "product_goal": "Tro ly tu hoc co suy luan dua tren tai lieu ca
     nhan",
  "users": ["student", "self-learner"],
  "core_use_cases": [
    "definition_qa",
    "comparison_qa",
    "why_reasoning",
    "exercise_support",
    "chapter_summary"
  ],
  "out_of_scope_v1": ["web_search", "grading_full_essay"],
  "quality_rules": [
    "every_major_claim_needs_support",
    "answer_must_be_grounded",
    "insufficient_context_must_be_explicit"
  ]
}
```

Bước 2. Thiết kế mô hình dữ liệu toàn hệ thống

Mục tiêu của bước: Tạo ra một data contract thống nhất để mọi module trao đổi dữ liệu rõ ràng, không hiểu khác nhau.

Input: Yêu cầu sản phẩm từ bước 1.
Xử lý chính: Thiết kế schema cho document, chunk, user query, query plan, evidence set, answer, feedback.
Output: Bộ schema JSON/Pydantic/ORM dùng xuyên suốt dự án.
Output này trở thành input cho: Bước 3 – Thiết kế project skeleton và hạ tầng.
Module / file nên có: `app/schemas/*.py`, `docs/data_contracts.md`, `app/db/models.py`

Các entity bắt buộc nên có

- **Document**: đại diện cho một file tài liệu gốc.
- **Chunk**: một đoạn nội dung đã chuẩn hóa, có metadata đầy đủ.
- **Concept**: một khái niệm học thuật được trích ra từ chunk.
- **QueryRequest**: gói thông tin truy vấn online.
- **QueryPlan**: kế hoạch suy luận gồm type, sub-questions, filters.
- **EvidenceItem**: bằng chứng được chọn sau retrieve và rerank.
- **AnswerRecord**: câu trả lời cuối cùng và citations.
- **FeedbackRecord**: phản hồi của user sau khi xem answer.

Ví dụ schema document

```
{
  "document_id": "doc_001",
  "user_id": "user_001",
  "subject": "database",
  "title": "Normalization Slides",
  "source_type": "slides",
  "language": "vi",
  "file_path": "storage/user_001/database/norm_slides.pdf",
  "uploaded_at": "2026-04-05T10:00:00"
}
```

Ví dụ schema chunk

```
{
  "chunk_id": "chunk_0001",
  "document_id": "doc_001",
  "user_id": "user_001",
  "subject": "database",
  "chapter": "Normalization",
  "section": "BCNF",
```

```
  "page_start": 12,
  "page_end": 13,
  "content_type": "definition",
  "text": "...",
  "token_count": 420,
  "keywords": ["BCNF", "FD", "dependency preservation"]
}
```

Bước 3. Thiết kế cấu trúc project, storage và service map

Mục tiêu của bước: Biến schema ở bước 2 thành một codebase có tổ chức, dễ mở rộng.
Input: Schema dữ liệu, lựa chọn công nghệ, yêu cầu triển khai.
Xử lý chính: Phân tách backend thành các module ingestion, retrieval, reasoning, verification, evaluation và API. Đồng thời quyết định nơi lưu file gốc, nơi lưu metadata, nơi lưu vector index.
Output: Project skeleton và sơ đồ dịch vụ.
Output này trở thành input cho: Bước 4 – Bắt đầu đường ống upload/lưu file gốc.
Module / file nên có: `app/`, `storage/`, `scripts/`, `docker-compose.yml`, `README.md`

Khuyến nghị cấu trúc thư mục

```
rag-learning-assistant/
  app/
    api/
    core/
    db/
    ingestion/
    retrieval/
    reasoning/
    verification/
    evaluation/
  storage/
  scripts/
  tests/
  docs/
  docker-compose.yml
```

Service map tối thiểu

- **FastAPI**: phục vụ API.
- **PostgreSQL hoặc SQLite**: lưu metadata và logs.

- **Qdrant**: lưu vector dense/sparse và payload.
- **File storage**: local hoặc MinIO/S3.

4.1 Cấu trúc dự án dự kiến trên GitHub

Phần này mở rộng trực tiếp cho Bước 3. Mục tiêu là biến "project skeleton" ở mức ý tưởng thành một **repository GitHub có thể clone về và bắt đầu code ngay**. Cấu trúc dưới đây đi theo nguyên tắc tách rõ **miền nghiệp vụ**, **dịch vụ chạy nền**, **tài liệu kỹ thuật** và **hạ tầng triển khai**.

Bổ sung cho Bước 3. Cấu trúc repository GitHub đề xuất

Mục tiêu của bước: Đưa ra một cấu trúc repository đủ chi tiết để nhóm phát triển có thể phân công việc, review code và triển khai CI/CD rõ ràng.
Input: Schema dữ liệu, service map, pipeline ingestion và pipeline reasoning đã chốt ở các bước trước.
Xử lý chính: Tách repository thành các lớp: ứng dụng chính, scripts vận hành, tài liệu thiết kế, test, cấu hình môi trường, và thư mục triển khai. Quy định rõ nơi đặt business logic, nơi đặt adapters cho database/vector store, nơi đặt prompt, và nơi đặt mã đánh giá.
Output: Một project tree chuẩn để đưa lên GitHub, kèm mô tả trách nhiệm từng thư mục và từng file quan trọng.
Output này trở thành input cho: Các bước từ Bước 4 trở đi sẽ hiện thực hóa lần lượt các thư mục và file trong project tree này.
Module / file nên có: `README.md`, `pyproject.toml` hoặc `requirements.txt`, `app/`, `scripts/`, `tests/`, `docs/`, `deploy/`, `docker-compose.yml`, `.env.example`, `.github/workflows/*.yml`

Project tree đề xuất trên GitHub

```
tutorrag/
  README.md
  LICENSE
  .gitignore
  .env.example
  pyproject.toml
  requirements.txt
  docker-compose.yml
  Dockerfile

  .github/
    workflows/
      ci.yml
```

```
    lint.yml
    test.yml

docs/
  problem_statement.md
  requirements.md
  architecture.md
  data_contracts.md
  api_spec.md
  evaluation_plan.md
  deployment_guide.md
  diagrams/
    overall_pipeline.png
    ingestion_pipeline.png
    online_reasoning_pipeline.png

app/
  main.py

  api/
    __init__.py
    deps.py
    routes_health.py
    routes_documents.py
    routes_chat.py
    routes_search.py
    routes_admin.py

  core/
    __init__.py
    config.py
    logging.py
    constants.py
    exceptions.py
    security.py

  schemas/
    __init__.py
    document.py
    chunk.py
    concept.py
    query.py
    evidence.py
    answer.py
    feedback.py
```

```
db/
  __init__.py
  session.py
  base.py
  models.py
  repositories/
    document_repo.py
    chunk_repo.py
    answer_repo.py
    feedback_repo.py

storage/
  __init__.py
  file_manager.py
  local_storage.py
  s3_storage.py

ingestion/
  __init__.py
  loaders.py
  parser_pdf.py
  parser_docx.py
  parser_pptx.py
  cleaner.py
  normalizer.py
  chunker.py
  metadata_builder.py
  embedding_jobs.py
  indexer.py

knowledge/
  __init__.py
  concept_extractor.py
  concept_index.py
  relation_graph.py
  prerequisite_graph.py
  keyword_registry.py

retrieval/
  __init__.py
  embedder.py
  sparse_encoder.py
  vector_store.py
  metadata_filters.py
```

```
    dense_retriever.py
    sparse_retriever.py
    hybrid_retriever.py
    reranker.py
    query_expander.py
    query_classifier.py
    query_decomposer.py
    evidence_builder.py

  prompts/
    system_prompts.py
    reasoning_prompts.py
    verification_prompts.py
    study_mode_prompts.py

  reasoning/
    __init__.py
    planner.py
    synthesizer.py
    answer_generator.py
    study_mode_router.py
    citation_builder.py

  verification/
    __init__.py
    claim_checker.py
    support_checker.py
    answer_verifier.py
    hallucination_guard.py

  services/
    __init__.py
    document_service.py
    ingestion_service.py
    retrieval_service.py
    tutoring_service.py
    evaluation_service.py

  orchestration/
    __init__.py
    offline_pipeline.py
    online_pipeline.py
    job_dispatcher.py

  evaluation/
```

```
    __init__.py
    dataset_builder.py
    metrics.py
    ragas_runner.py
    benchmark.py
    error_analysis.py

  utils/
    __init__.py
    token_utils.py
    text_utils.py
    time_utils.py
    id_utils.py

scripts/
  ingest_folder.py
  reindex_subject.py
  rebuild_concepts.py
  run_local_eval.py
  export_test_dataset.py
  bootstrap_dev.sh

tests/
  conftest.py
  unit/
    test_chunker.py
    test_query_classifier.py
    test_retriever.py
    test_reranker.py
    test_answer_verifier.py
  integration/
    test_document_upload_api.py
    test_ingestion_pipeline.py
    test_online_pipeline.py
  e2e/
    test_user_question_flow.py

storage/
  .gitkeep

data/
  samples/
    sample_notes/
    sample_slides/
    sample_exams/
```

```
notebooks/
  retrieval_experiments.ipynb
  embedding_analysis.ipynb
  evaluation_debug.ipynb

deploy/
  nginx.conf
  qdrant/
    collection_setup.json
  postgres/
    init.sql
  systemd/
    tutorrag.service
```

Giải thích trách nhiệm từng thư mục chính

- **`app/`**: toàn bộ mã nguồn backend chính. Đây là trái tim của dự án.
- **`app/api/`**: định nghĩa endpoint FastAPI. Thư mục này chỉ nên làm nhiệm vụ nhận request, validate sơ bộ, gọi service và trả response.
- **`app/core/`**: cấu hình toàn cục, logging, constants, exception chuẩn. Không đặt business logic nặng ở đây.
- **`app/schemas/`**: các Pydantic schema hoặc data models dùng để truyền dữ liệu giữa các tầng.
- **`app/db/`**: ORM models, session và repository cho metadata database như PostgreSQL.
- **`app/storage/`**: adapter lưu file gốc. Có thể bắt đầu bằng local storage, sau nâng lên S3/MinIO.
- **`app/ingestion/`**: mọi logic offline từ parse, clean, chunk, build metadata đến index.
- **`app/knowledge/`**: lớp tri thức bổ sung như concept index, relation graph, prerequisite graph.
- **`app/retrieval/`**: dense retrieval, sparse retrieval, hybrid retrieval, metadata filter, query decomposition và evidence building.
- **`app/prompts/`**: nơi quản lý prompt một cách tập trung để dễ versioning và review.
- **`app/reasoning/`**: planner, synthesizer, answer generator; đây là tầng suy luận dựa trên evidence.

- **`app/verification/`**: kiểm tra claim, support, citation và phát hiện câu trả lời vượt ra ngoài evidence.
- **`app/services/`**: tầng service ghép business logic cho API gọi tới. Đây là nơi phối hợp nhiều module cùng lúc.
- **`app/orchestration/`**: pipeline end-to-end, đặc biệt hữu ích khi bạn muốn một entry point rõ ràng cho offline ingestion và online reasoning.
- **`app/evaluation/`**: mã đánh giá chất lượng retrieval, reasoning, grounding và benchmark.
- **`scripts/`**: các lệnh tiện ích để vận hành, reindex, chạy evaluation, bootstrap môi trường dev.
- **`tests/`**: toàn bộ test unit, integration và end-to-end.
- **`docs/`**: tài liệu kỹ thuật để cả nhóm đọc trên GitHub mà không cần mở code.
- **`deploy/`**: cấu hình triển khai thực tế như Nginx, PostgreSQL init script, systemd, Qdrant setup.
- **`.github/workflows/`**: pipeline CI để lint, test và kiểm tra build mỗi khi push hoặc tạo pull request.

Những file gốc nên có ngay từ commit đầu tiên

1. `README.md`: mô tả dự án, cách chạy local, kiến trúc tổng quan, roadmap.
2. `.env.example`: danh sách biến môi trường mẫu như database URL, Qdrant URL, API key.
3. `docker-compose.yml`: khởi tạo local stack gồm API, Qdrant, PostgreSQL.
4. `pyproject.toml` hoặc `requirements.txt`: quản lý dependency.
5. `docs/architecture.md`: sơ đồ ingest pipeline và online pipeline.
6. `tests/`: bộ test tối thiểu để chống gãy luồng chính khi refactor.

Nguyên tắc đặt code trong repository

- API layer không được chứa toàn bộ business logic.
- Prompt không hard-code rải rác trong service hoặc route; phải gom vào thư mục `app/prompts/`.

- Adapter cho Qdrant, PostgreSQL, storage phải được bọc trong module riêng để sau này thay thế công nghệ dễ hơn.
- Retrieval logic và reasoning logic phải tách nhau; retrieval không tự sinh câu trả lời.
- Verification phải là một bước độc lập thay vì trộn vào answer generator.

Mapping giữa thư mục và pipeline nghiệp vụ

Giai đoạn nghiệp vụ	**Thư mục chính**	**File tiêu biểu**
Upload và lưu file gốc	`app/api`, `app/storage`, `app/db`	`routes_documents.py`, `file_manager.py`, `document_repo.py`
Parse, clean, chunk, index	`app/ingestion`	`parser_pdf.py`, `cleaner.py`, `chunker.py`, `indexer.py`
Trích concept và xây graph	`app/knowledge`	`concept_extractor.py`, `relation_graph.py`
Hiểu truy vấn và retrieve	`app/retrieval`	`query_classifier.py`, `hybrid_retriever.py`, `reranker.py`
Suy luận và sinh câu trả lời	`app/reasoning`	`synthesizer.py`, `answer_generator.py`, `citation_builder.py`
Kiểm tra grounding	`app/verification`	`claim_checker.py`, `answer_verifier.py`
Đánh giá chất lượng	`app/evaluation`, `scripts`	`ragas_runner.py`, `benchmark.py`, `run_local_eval.py`
Triển khai và CI	`deploy`, `.github/workflows`	`nginx.conf`, `ci.yml`, `test.yml`

Luồng làm việc GitHub khuyến nghị

1. Nhánh `main`: luôn ở trạng thái deploy được.
2. Nhánh `develop` hoặc làm việc trực tiếp bằng feature branches nếu nhóm nhỏ.
3. Mỗi tính năng lớn đi theo một nhánh riêng, ví dụ: `feature/document-upload`, `feature/hybrid-retrieval`, `feature/answer-verification`.
4. Mỗi pull request cần đi kèm:
   mô tả thay đổi, ảnh hưởng đến pipeline nào, test nào đã chạy.
5. CI tối thiểu phải chạy lint + unit test + integration smoke test.

Output mẫu của bước bổ sung này

```
{
  "repository_name": "tutorrag",
  "main_directories": [
    "app",
    "docs",
    "scripts",
    "tests",
    "deploy",
    ".github/workflows"
  ],
  "core_modules": [
    "ingestion",
    "knowledge",
    "retrieval",
    "reasoning",
    "verification",
    "evaluation"
  ],
  "entrypoints": [
    "app/main.py",
    "scripts/ingest_folder.py",
    "scripts/run_local_eval.py"
  ]
}
```

5 Giai đoạn B – Xây dựng knowledge base từ tài liệu gốc

Bước 4. Upload tài liệu và lưu file gốc

Mục tiêu của bước: Nhận file từ người dùng và lưu an toàn trước khi bắt đầu parse.
Input: File nhị phân do user tải lên + metadata ban đầu như môn học, title, loại tài liệu.
Xử lý chính: Kiểm tra định dạng file, tạo document_id, lưu file vào storage, ghi bản ghi document vào DB.
Output: File gốc nằm trong storage và một bản ghi document ở metadata DB.
Output này trở thành input cho: Bước 5 – Parse nội dung tài liệu.
Module / file nên có: `app/api/routes_documents.py`, `app/ingestion/storage.py`, `app/db/repositories.py`

Checklist xử lý

1. Kiểm tra phần mở rộng cho phép: PDF/DOCX/PPTX/TXT/MD.
2. Tạo `document_id` duy nhất.
3. Chuẩn hóa tên file để tránh ký tự lạ.
4. Lưu file vào đường dẫn theo user và môn.
5. Ghi metadata document, trạng thái ban đầu là `uploaded`.

Output mẫu

```
{
  "document_id": "doc_001",
  "status": "uploaded",
  "storage_path": "storage/user_001/database/norm_slides.pdf",
  "subject": "database"
}
```

Bước 5. Parse tài liệu thành văn bản có cấu trúc

Mục tiêu của bước: Trích nội dung văn bản, tiêu đề, số trang, block, heading, bảng và vùng công thức từ file gốc theo đúng thứ tự ngữ nghĩa.
Input: File gốc từ bước 4 và metadata document tương ứng.
Xử lý chính: Chọn parser theo loại file. Với PDF học thuật, ưu tiên parser có hiểu layout như Docling hoặc pipeline parse có awareness về

block/bảng/công thức; chỉ rơi về text parser thuần khi tài liệu đơn giản. Với DOCX/PPTX thì đọc cấu trúc heading, bullet, bảng. Với file scan hoặc ảnh chụp, thêm bước OCR rồi mới ghép lại theo reading order.
Output: Parsed document gồm text thô, blocks theo trang, vùng layout quan trọng và metadata layout cơ bản.
Output này trở thành input cho: Bước 6 – Normalize nội dung.
Module / file nên có: `app/ingestion/parser_docling.py`, `app/ingestion/parser_pdf.py`, `app/ingestion/parser_docx.py`, `app/ingestion/parser_pptx.py`, `app/ingestion/parser_ocr.py`

Bạn nên trích những gì từ parser

- Text của từng trang.
- Danh sách block hoặc paragraph.
- Heading nếu nhận diện được.
- Số trang bắt đầu và kết thúc của mỗi block.
- Chỉ báo nếu block là bảng, công thức, bullet list, ví dụ.
- Tọa độ hoặc vùng tham chiếu của block để về sau trace lại nguồn gốc khi verifier cần kiểm tra.

Mẫu output parse

```
{
  "document_id": "doc_001",
  "pages": [
    {
      "page_number": 12,
      "blocks": [
        {"type": "heading", "text": "BCNF"},
        {"type": "paragraph", "text": "A relation schema R ..."}
      ]
    }
  ]
}
```

Nếu parser lấy text sai thứ tự đọc, bước chunking sau đó sẽ hỏng. Vì vậy bước parse không chỉ cần "lấy được chữ", mà phải giữ được **thứ tự ngữ nghĩa** của nội dung.

Với tài liệu Bách Khoa có nhiều công thức toán, sơ đồ và bảng, parse text thuần túy rất dễ làm hỏng ngữ nghĩa. Vì vậy nên lưu song song **text đã parse**, **loại block** và **vị trí block trên trang** để về sau có thể kiểm tra lại câu trả lời hoặc hiển thị vùng nguồn gốc chính xác.

Bước 6. Normalize nội dung và loại nhiễu

Mục tiêu của bước: Làm sạch nội dung để chunking và embedding không bị nhiễu bởi header/footer, số trang và lỗi định dạng, nhưng vẫn giữ được ký hiệu và cấu trúc học thuật quan trọng.
Input: Parsed document từ bước 5.
Xử lý chính: Xóa header/footer lặp, chuẩn hóa whitespace, gộp các dòng bị ngắt sai, giữ heading và các thực thể học thuật. Với công thức, ký hiệu đặc biệt và tên biến, chỉ normalize nhẹ; không được biến đổi đến mức làm mất nghĩa.
Output: Clean document có cấu trúc rõ ràng, ít nhiễu và vẫn bảo toàn nội dung chuyên ngành.
Output này trở thành input cho: Bước 7 – Chunking theo cấu trúc.
Module / file nên có: `app/ingestion/cleaner.py`, `app/ingestion/normalizer.py`, `app/ingestion/symbol_preserver.py`

Các thao tác normalize nên có

1. Xóa chuỗi lặp ở đầu/trang cuối như tên trường, môn học, page number.
2. Chuẩn hóa khoảng trắng, xuống dòng, tab.
3. Gộp các đoạn ngắt dòng giữa câu do layout PDF.
4. Giữ lại các heading, bullet và định dạng danh sách.
5. Đánh dấu vùng là “definition”, “example”, “theorem”, “exercise” nếu có thể suy ra từ heading hoặc pattern.
6. Bảo toàn ký hiệu toán học, ký hiệu logic, tên biến và chuỗi viết tắt chuyên ngành ở bản text chính hoặc ở một trường song song.

Input và output của bước này cần khác nhau ra sao

- **Input**: text còn lẫn thành phần trình bày.
- **Output**: text hướng nội dung, ít phụ thuộc layout, nhưng không đánh mất công thức hoặc ký hiệu gốc.

Bước 7. Chia chunk theo cấu trúc học thuật

Mục tiêu của bước: Tạo các chunk đủ ngắn để retrieve tốt nhưng vẫn đủ giàu ngữ cảnh để LLM hiểu đúng.
Input: Clean document từ bước 6.
Xử lý chính: Chia trước theo chapter/section/heading; sau đó nếu đoạn còn quá dài thì cắt tiếp theo token budget với overlap hợp lý.
Output: Danh sách chunks thô có text và ranh giới trang/section.
Output này trở thành input cho: Bước 8 – Gắn metadata và phân loại chunk.
Module / file nên có: `app/ingestion/chunker.py`, `app/ingestion/tokenizer.py`

Nguyên tắc chunking cho bài toán tự học

- Không cắt giữa định nghĩa và ví dụ minh họa nếu hai phần nằm sát nhau.
- Không cắt giữa statement của định lý và phần giải thích ngắn ngay sau nó.
- Ưu tiên 400–700 tokens/chunk với overlap 80–120 tokens.
- Nếu là slide, có thể chunk theo từng slide hoặc theo mỗi nhóm bullet liên quan.

Output mẫu

```
[
  {
    "chunk_id": "temp_001",
    "page_start": 12,
    "page_end": 13,
    "section": "BCNF",
    "text": "Boyce-Codd Normal Form ..."
  }
]
```

Bước 8. Gắn metadata cho từng chunk

Mục tiêu của bước: Làm cho mỗi chunk trở thành một đơn vị tri thức có thể lọc, truy hồi, giải thích và log được.
Input: Chunks thô từ bước 7 + document metadata từ bước 4.
Xử lý chính: Gắn user_id, subject, chapter, section, content_type, keywords, page range, title, source_type, language cho từng chunk.
Output: Chunk records đầy đủ metadata.

Output này trở thành input cho: Bước 9 – Sinh embeddings dense.
Module / file nên có: `app/ingestion/metadata.py`, `app/ingestion/enrichers.py`

Các trường metadata rất đáng giá cho retrieval về sau

- `subject`: lọc theo môn học.
- `chapter`, `section`: lọc theo chương/mục.
- `content_type`: definition, example, theorem, proof, exercise, solution.
- `difficulty`: easy, medium, hard.
- `keywords`: từ khóa học thuật quan trọng.

Bước 9. Sinh dense embeddings cho chunk

Mục tiêu của bước: Mã hóa ngữ nghĩa của chunk thành vector để phục vụ semantic retrieval.
Input: Chunk records đã hoàn chỉnh từ bước 8.
Xử lý chính: Đưa text của mỗi chunk qua mô hình embedding, đồng thời giữ liên kết giữa vector và metadata gốc.
Output: Dense vectors cho từng chunk.
Output này trở thành input cho: Bước 10 – Tạo sparse representation; Bước 13 – Upsert vào vector DB.
Module / file nên có: `app/retrieval/embedder.py`, `app/retrieval/embedding_jobs.py`

Điểm cần lưu ý

1. Chỉ embed phần text nội dung chính, không embed metadata rác.
2. Nếu dùng batching, phải log chunk nào thất bại để retry.
3. Nên cache embedding theo hash của text để tránh tính lại khi reindex.

Output mẫu

```
{
  "chunk_id": "chunk_0001",
  "dense_vector": [0.014, -0.082, ...],
  "embedding_model": "text-embedding-3-small"
}
```

Bước 10. Tạo chỉ mục sparse / lexical representation

Mục tiêu của bước: Bổ sung khả năng tìm theo từ khóa chính xác, viết tắt, tên thuật toán, ký hiệu học thuật.
Input: Chunk records từ bước 8.
Xử lý chính: Tạo sparse vector hoặc chỉ mục BM25/term-weight cho từng chunk.
Output: Sparse representation hoặc thông tin cần thiết cho lexical search.
Output này trở thành input cho: Bước 13 – Upsert vào vector DB; Bước 17 – Hybrid retrieval.
Module / file nên có: `app/retrieval/sparse_index.py`, `app/retrieval/bm25.py`

Tại sao bước này quan trọng

- Người học thường hỏi các từ khóa chính xác như BCNF, DFS, MLE, entropy, backprop.
- Dense retrieval tốt ở ngữ nghĩa nhưng có thể bỏ sót keyword hiếm.
- Sparse retrieval giúp bắt trúng tên khái niệm, công thức, ký hiệu.

6 Giai đoạn C – Làm knowledge base thông minh hơn vector search thuần túy

Bước 11. Xây concept index

Mục tiêu của bước: Biến các đoạn text thành lớp khái niệm có cấu trúc để hỗ trợ reasoning, giải thích, query expansion và làm nền cho GraphRAG ở mức nhẹ.
Input: Chunk records từ bước 8.
Xử lý chính: Trích các khái niệm, định nghĩa, thuật ngữ, công thức, tên thuật toán từ chunk; lưu chúng thành một index riêng. Có thể bắt đầu bằng JSON/SQL đơn giản trước khi nâng cấp thành graph store chuyên dụng.
Output: Danh sách concept records liên kết ngược về chunk nguồn.
Output này trở thành input cho: Bước 12 – Xây relation graph; Bước 16 – Query decomposition có thể dùng concept index để mở rộng truy vấn.
Module / file nên có: `app/knowledge/concept_extractor.py`, `app/knowledge/concept_index.py`, `app/knowledge/concept_store.py`

Ví dụ concept record

```
{
```

```
  "concept_id": "concept_010",
  "name": "BCNF",
  "aliases": ["Boyce-Codd Normal Form"],
  "subject": "database",
  "definition_chunk_ids": ["chunk_0001"],
  "related_keywords": ["functional dependency", "normalization"]
}
```

Lợi ích

- Hỗ trợ đồng nhất từ đồng nghĩa/viết tắt.
- Giúp query "chuẩn 3" vẫn tìm được "3NF".
- Làm nền cho sơ đồ prerequisite hoặc quan hệ khái niệm.

Bước 12. Xây relation graph giữa khái niệm, ví dụ và prerequisite

Mục tiêu của bước: Tạo khả năng truy luận có cấu trúc thay vì chỉ dựa vào độ gần vector.
Input: Concept index từ bước 11 + chunk records từ bước 8.
Xử lý chính: Tạo các cạnh như prerequisite, contrast, example-of, theorem-for, used-in-solution, same-topic. Nếu cần hỏi đáp theo prerequisite hoặc learning path, có thể lưu graph này trong Neo4j hoặc một graph store nhỏ; không nên xem metadata filter trong vector DB là graph traversal hoàn chỉnh.
Output: Đồ thị quan hệ tri thức.
Output này trở thành input cho: Bước 16 – Query decomposition; Bước 18 và 19 – Evidence building và reasoning synthesis.
Module / file nên có: `app/knowledge/relation_graph.py`, `app/knowledge/graph_builders.py`, `app/knowledge/graph_store.py`

Ví dụ cạnh trong graph

```
{
  "from": "concept_BCNF",
  "to": "concept_3NF",
  "relation": "stronger_than"
}
```

Lợi ích của relation graph

- Câu hỏi so sánh được hỗ trợ tốt hơn.
- Câu hỏi "tại sao" có thể kéo thêm khái niệm prerequisite.
- Câu hỏi bài tập có thể tìm ví dụ tương tự nhanh hơn.

- Hỗ trợ trực tiếp các câu như "Cần học gì trước khi học BCNF?" hoặc "Khái niệm nào là nền cho phần này?".

Bước 13. Upsert toàn bộ tri thức vào vector database

Mục tiêu của bước: Đưa dense vector, sparse vector và metadata vào cùng một collection để phục vụ hybrid retrieval.
Input: Dense vectors từ bước 9, sparse representation từ bước 10, metadata từ bước 8.
Xử lý chính: Tạo collection, định nghĩa payload schema, upsert từng chunk, lưu mapping giữa chunk_id và vector point.
Output: Knowledge base truy hồi được.
Output này trở thành input cho: Bước 14 – Xử lý query online.
Module / file nên có: `app/retrieval/vector_store.py`, `scripts/reindex_subject.py`

Payload tối thiểu nên lưu

```
{
  "chunk_id": "chunk_0001",
  "document_id": "doc_001",
  "user_id": "user_001",
  "subject": "database",
  "chapter": "Normalization",
  "section": "BCNF",
  "page_start": 12,
  "page_end": 13,
  "content_type": "definition",
  "title": "Normalization Slides"
}
```

Lưu ý vận hành

- Mọi truy hồi online phải luôn filter theo `user_id`.
- Nếu user đã chọn môn học, nên filter thêm theo `subject`.
- Reindex nên chạy được theo document, theo subject hoặc toàn user.

7 Giai đoạn D – Online pipeline cho câu hỏi có suy luận

Bước 14. Nhận query online và chuẩn hóa request

Mục tiêu của bước: Biến câu hỏi thô của user thành một gói truy vấn giàu ngữ cảnh.
Input: Câu hỏi user, session history ngắn, subject đang chọn trên UI, chế độ học nếu có.
Xử lý chính: Chuẩn hóa chuỗi hỏi, lưu session id, xác định subject hiện tại, thu gọn lịch sử chat cần thiết.
Output: QueryRequest chuẩn hóa.
Output này trở thành input cho: Bước 15 – Query understanding.
Module / file nên có: `app/api/routes_chat.py`, `app/reasoning/query_request.py`, `app/core/session.py`

Ví dụ QueryRequest

```
{
  "session_id": "sess_001",
  "user_id": "user_001",
  "question": "Tai sao trong bai nay chap nhan 3NF thay vi BCNF?",
  "subject_hint": "database",
  "study_mode": "explain",
  "chat_history": ["..."]
}
```

Bước 15. Query understanding và phân loại intent

Mục tiêu của bước: Xác định loại câu hỏi để chọn chiến lược retrieve và reasoning phù hợp.
Input: QueryRequest từ bước 14.
Xử lý chính: Phân loại câu hỏi thành lookup, explanation, comparison, why-question, problem-solving, contradiction-checking, summary, quiz-generation. Đồng thời suy ra mức độ cần multi-hop reasoning và quyết định có cần planner kiểu agent có giới hạn hay không.
Output: QueryAnalysis gồm query type, intent, filters, priority content types, difficulty level.
Output này trở thành input cho: Bước 16 – Query decomposition.
Module / file nên có: `app/reasoning/query_classifier.py`, `app/reasoning/query_analysis.py`

Ví dụ output

```
{
  "query_type": "why_reasoning",
  "requires_multi_hop": true,
  "subject": "database",
  "preferred_content_types": ["definition", "example", "solution"],
  "reasoning_depth": "medium"
}
```

Bước 16. Query decomposition và lập kế hoạch suy luận

Mục tiêu của bước: Tách câu hỏi lớn thành các câu hỏi con có thể truy hồi và kiểm chứng được.
Input: QueryAnalysis từ bước 15, query gốc, concept index và relation graph.
Xử lý chính: Sinh sub-questions, sub-queries, giả thuyết cần kiểm tra, metadata filters và thứ tự ưu tiên truy hồi. Nếu dùng agentic planning, chỉ nên dùng planner kiểu ReAct có giới hạn: số vòng ít, số tool ít và có điều kiện dừng rõ ràng.
Output: QueryPlan.
Output này trở thành input cho: Bước 17 – Hybrid retrieval nhiều vòng.
Module / file nên có: `app/reasoning/query_decomposer.py`, `app/reasoning/planner.py`, `app/reasoning/react_planner.py`

Ví dụ QueryPlan

```
{
  "main_question": "Tai sao chap nhan 3NF thay vi BCNF?",
  "sub_questions": [
    "3NF la gi?",
    "BCNF la gi?",
    "Trong bai hien tai co trade-off dependency preservation khong?"
  ],
  "filters": {
    "user_id": "user_001",
    "subject": "database"
  }
}
```

Nguyên tắc decomposition tốt

- Mỗi sub-question phải đủ cụ thể để retrieve được.
- Không tạo quá nhiều sub-question không cần thiết vì sẽ tăng độ trễ.

- Với câu hỏi bài tập, nên có ít nhất một sub-question hỏi về **dữ kiện đề bài** và một sub-question hỏi về **nguyên lý liên quan**.
- Nếu dùng planner dạng agent, phải đặt giới hạn số vòng reasoning/retrieval và có tiêu chí dừng như "đã đủ evidence" hoặc "đã chạm ngưỡng lặp tối đa".

Bước 17. Hybrid retrieval nhiều vòng

Mục tiêu của bước: Lấy đủ bằng chứng từ knowledge base cho từng sub-question, thay vì chỉ retrieve một lần cho câu hỏi gốc.
Input: QueryPlan từ bước 16, vector DB từ bước 13, concept index và metadata filters.
Xử lý chính: Chạy dense retrieval, sparse retrieval và retrieval theo metadata/concept. Có thể lặp lại vòng retrieval nếu reasoning sơ bộ cho thấy evidence còn thiếu, nhưng phải giới hạn số vòng để tránh agent loop vô ích.
Output: Candidate evidence pool.
Output này trở thành input cho: Bước 18 – Merge, dedupe, rerank.
Module / file nên có: `app/retrieval/retriever.py`, `app/retrieval/hybrid_retriever.py`, `app/retrieval/query_expander.py`

Chi tiết nên làm trong bước này

1. Với mỗi sub-query, tạo embedding query.
2. Chạy dense top-k.
3. Chạy sparse/BM25 top-k.
4. Thêm retrieval theo concept hoặc keyword alias nếu có.
5. Hợp nhất kết quả và gắn nguồn sub-query nào đã lấy ra kết quả đó.

Output mẫu

```
[
  {
    "chunk_id": "chunk_0001",
    "retrieved_for": "BCNF la gi?",
    "dense_score": 0.81,
    "sparse_score": 0.66
  },
  {
    "chunk_id": "chunk_0045",
    "retrieved_for": "trade-off dependency preservation",
    "dense_score": 0.77,
```

```
    "sparse_score": 0.71
  }
]
```

Bước 18. Merge, deduplicate và rerank thành evidence set

Mục tiêu của bước: Rút candidate pool thành tập bằng chứng mạnh nhất, ít trùng lặp nhất và phủ đủ các ý cần suy luận.
Input: Candidate evidence pool từ bước 17.
Xử lý chính: Gộp các chunk trùng, tính điểm tổng hợp, dùng reranker chấm relevance theo cặp (question, chunk), rồi chọn top evidence có độ phủ tốt.
Output: Evidence set đã xếp hạng.
Output này trở thành input cho: Bước 19 – Reasoning synthesis.
Module / file nên có: `app/retrieval/reranker.py`, `app/reasoning/evidence_builder.py`

Điều bắt buộc ở bước này

- Không chỉ lấy "điểm cao nhất" mà phải kiểm tra **độ phủ** giữa các sub-question.
- Hạn chế nhét nhiều chunk lặp ý giống nhau.
- Gắn lý do giữ lại mỗi evidence: định nghĩa, ví dụ, lời giải, phản ví dụ.

Ví dụ output

```
{
  "evidence_items": [
    {
      "chunk_id": "chunk_0001",
      "role": "definition_of_BCNF",
      "score": 0.93
    },
    {
      "chunk_id": "chunk_0045",
      "role": "tradeoff_example",
      "score": 0.91
    }
  ]
}
```

Bước 19. Reasoning synthesis trên evidence set

Mục tiêu của bước: Tạo câu trả lời biết kết nối nhiều bằng chứng nhưng vẫn bám chặt vào chúng.
Input: Question gốc, QueryAnalysis, Evidence set đã rerank.
Xử lý chính: Cho LLM tổng hợp bằng chứng, rút ra các claims chính, viết reasoning summary, tạo draft answer và gắn citations theo từng claim quan trọng.
Output: Draft answer có structure rõ ràng.
Output này trở thành input cho: Bước 20 – Verification.
Module / file nên có: `app/reasoning/synthesizer.py`, `app/generation/prompts.py`, `app/generation/answerer.py`

Luật prompt quan trọng

1. Chỉ suy luận từ evidence được đưa vào.
2. Mọi kết luận chính phải chỉ ra chunk hỗ trợ.
3. Nếu evidence mâu thuẫn, phải nêu điểm mâu thuẫn thay vì ép ra một kết luận giả.
4. Nếu evidence chưa đủ, phải trả lời thiếu bằng chứng.

Output mẫu

```
{
  "direct_answer": "...",
  "reasoning_summary": [
    "BCNF la dang manh hon 3NF.",
    "Tuy nhien decomposition len BCNF co the lam mat dependency
       preservation.",
    "Tai lieu dang xet uu tien bao toan phu thuoc ham."
  ],
  "citations": ["chunk_0001", "chunk_0045"]
}
```

Bước 20. Verification và claim checking

Mục tiêu của bước: Ngăn câu trả lời cuối cùng vượt ra ngoài evidence hoặc trích dẫn sai chỗ.
Input: Draft answer từ bước 19 và evidence set từ bước 18.
Xử lý chính: Tách answer thành claims, kiểm tra từng claim có được support bởi evidence hay không, loại bỏ hoặc viết lại claim không được hỗ trợ. Online verifier phải chạy trong runtime; còn các framework như Ragas nên dùng chủ yếu cho evaluation định kỳ và benchmark faithfulness ở bước

23.
Output: Final answer đã kiểm chứng.
Output này trở thành input cho: Bước 21 – Logging và feedback.
Module / file nên có: `app/verification/verifier.py`, `app/verification/claim_checker.py`, `app/evaluation/ragas_runner.py`

Bộ câu hỏi verifier nên tự hỏi

- Claim này có evidence trực tiếp không?
- Citation gắn có thực sự support claim đó không?
- Có đoạn nào answer suy diễn quá xa không?
- Có sub-question nào trong QueryPlan chưa được cover không?
- Nếu chấm theo faithfulness, câu trả lời này có câu nào không bám vào ngữ cảnh đã retrieve hay không?

Ví dụ output

```
{
  "verified": true,
  "removed_claims": [],
  "confidence": 0.84,
  "final_answer": "..."
}
```

Bước 21. Logging, analytics và feedback loop

Mục tiêu của bước: Biến mỗi phiên hỏi đáp thành dữ liệu để đánh giá và cải thiện hệ thống.
Input: Final answer, evidence set, query info, phản hồi user nếu có.
Xử lý chính: Lưu toàn bộ các trường quan trọng: question, query type, retrieved chunks, rerank scores, final citations, response time, feedback.
Output: Answer log và feedback records.
Output này trở thành input cho: Bước 23 – Evaluation định kỳ và cải thiện retrieval/reasoning.
Module / file nên có: `app/db/logging.py`, `app/evaluation/log_reader.py`

Những gì nên log

- Question gốc.
- Query type và QueryPlan.

- Top candidates và evidence cuối cùng.
- Prompt version.
- Câu trả lời cuối cùng.
- Feedback thumbs up/down hoặc nhãn "hữu ích / không hữu ích".

8 Giai đoạn E – Orchestration, API, đánh giá và triển khai

Bước 22. Đóng gói thành API và orchestration layer

Mục tiêu của bước: Kết nối toàn bộ pipeline thành một backend có thể gọi từ web app hoặc app di động.
Input: Tất cả các module từ bước 4 đến bước 21.
Xử lý chính: Xây các API upload, index, ask, list documents, list subjects, feedback; tạo service layer điều phối thứ tự chạy của từng thành phần.
Output: Một backend có endpoint rõ ràng.
Output này trở thành input cho: Bước 23 – Evaluation; Bước 24 – Deployment.
Module / file nên có: `app/api/main.py`, `app/services/*.py`, `app/api/routes_*.py`

Các endpoint tối thiểu

```
POST /documents/upload
POST /documents/{document_id}/index
GET /subjects
GET /subjects/{subject}/documents
POST /chat/ask
POST /chat/feedback
GET /health
```

Dòng chảy của endpoint /chat/ask

1. Nhận question.
2. Tạo QueryRequest.
3. Query understanding.
4. Query decomposition.
5. Hybrid retrieval.

6. Rerank và build evidence set.
7. Reasoning synthesis.
8. Verification.
9. Trả final answer + citations.

Bước 23. Xây bộ evaluation để đo retrieval và reasoning

Mục tiêu của bước: Đánh giá hệ thống một cách có hệ thống thay vì chỉ "thấy có vẻ đúng".
Input: Knowledge base đã xây, logs từ bước 21, một tập câu hỏi chuẩn do bạn soạn.
Xử lý chính: Xây benchmark dataset, chấm hit-rate, context precision/recall, faithfulness, answer relevancy, latency, cost.
Output: Báo cáo chất lượng theo phiên bản hệ thống.
Output này trở thành input cho: Quay lại các bước 7, 10, 17, 18, 19 để tối ưu.
Module / file nên có: `app/evaluation/dataset_builder.py`, `app/evaluation/metrics.py`, `scripts/eval_rag.py`

Cách xây bộ test

- 20 câu định nghĩa.
- 20 câu so sánh.
- 20 câu why/how cần suy luận đa bước.
- 20 câu bài tập hoặc tình huống áp dụng.
- 20 câu dễ gây nhầm lẫn giữa hai khái niệm gần nhau.

Các metric nên có

- Retrieval hit rate@k.
- Context precision.
- Context recall.
- Faithfulness.
- Answer relevancy.
- Latency trung bình.

Bước 24. Triển khai local, staging và production

Mục tiêu của bước: Đưa hệ thống từ môi trường phát triển sang môi trường chạy ổn định.
Input: Backend hoàn chỉnh từ bước 22 và cấu hình vận hành.
Xử lý chính: Dockerize services, quản lý biến môi trường, mount storage, backup DB, monitoring logs, thiết lập CI/CD cơ bản.
Output: Môi trường triển khai chạy được và có thể mở rộng.
Output này trở thành input cho: Vòng lặp cải tiến tiếp theo dựa trên feedback thực tế.
Module / file nên có: `Dockerfile`, `docker-compose.yml`, `.env`, `deploy/`

Môi trường nên có

- **Local**: để phát triển và debug nhanh.
- **Staging**: để test với dữ liệu gần giống production.
- **Production**: có logging, backup, monitoring.

9 Mối liên hệ input-output giữa các bước

Quy tắc dây chuyền
Mỗi bước chỉ nên nhận **một input contract rõ ràng** và trả **một output contract rõ ràng**. Tránh thiết kế module "làm tất cả" vì rất khó debug khi chất lượng retrieval kém hay reasoning sai.

9.1 Chuỗi offline

```
Raw file
-> ParsedDocument
-> CleanDocument
-> ChunkList
-> EnrichedChunkList
-> DenseVectors + SparseVectors
-> ConceptIndex + RelationGraph
-> VectorDB Collection
```

9.2 Chuỗi online

```
UserQuestion
-> QueryRequest
-> QueryAnalysis
-> QueryPlan
-> CandidateEvidencePool
-> EvidenceSet
-> DraftAnswer
-> VerifiedAnswer
-> AnswerLog
```

10 Kế hoạch triển khai đề xuất theo tuần

Tuần	Việc nên hoàn thành
Tuần 1	Bước 1, 2, 3. Chốt use case, schema, project skeleton.
Tuần 2	Bước 4, 5, 6. Hoàn thành upload, parse, normalize.
Tuần 3	Bước 7, 8, 9, 10. Hoàn thành chunking và indexing dense/sparse.
Tuần 4	Bước 11, 12, 13. Thêm concept index, relation graph và upsert vector DB.
Tuần 5	Bước 14, 15, 16. Hoàn thành query understanding và decomposition.
Tuần 6	Bước 17, 18. Hoàn thành hybrid retrieval, reranker, evidence set.
Tuần 7	Bước 19, 20. Hoàn thành reasoning synthesis và verification.
Tuần 8	Bước 21, 22, 23, 24. Logging, evaluation, deployment, demo.

11 Definition of Done cho từng lớp hệ thống

11.1 Offline ingestion hoàn thành khi nào?

- Upload được ít nhất 3 loại tài liệu khác nhau.
- Parse giữ đúng phần lớn thứ tự ngữ nghĩa.
- Chunk có metadata đầy đủ, truy xuất được trang nguồn.
- Reindex một tài liệu không làm hỏng các tài liệu khác.

11.2 Retrieval hoàn thành khi nào?

- Câu hỏi định nghĩa đơn giản retrieve đúng trong top-k phần lớn thời gian.

- Câu hỏi keyword hiếm vẫn retrieve được nhờ sparse search.
- Có thể lọc theo user và subject chính xác.

11.3 Reasoning pipeline hoàn thành khi nào?

- Câu hỏi so sánh và why-question có thể trả lời bằng nhiều evidence.
- Khi evidence thiếu, hệ thống biết từ chối kết luận chắc chắn.
- Verification loại được claim không có support.

12 Những lỗi kiến trúc thường gặp cần tránh

1. Chỉ dùng dense retrieval và bỏ qua sparse search.
2. Chunk quá to khiến retrieve trúng mà vẫn không đọc nổi đúng ý.
3. Không lưu số trang nên không thể trích nguồn đúng.
4. Không phân loại query type nên mọi câu hỏi bị xử lý một kiểu.
5. Không có verifier nên answer dễ "nói thêm" ngoài evidence.
6. Không log retrieved chunks nên không biết retrieval sai hay reasoning sai.
7. Không có bộ evaluation nên tối ưu theo cảm giác.

13 Điều chỉnh kỹ thuật mở rộng theo góp ý triển khai

Phần này tổng hợp các góp ý kỹ thuật quan trọng cho đồ án và chuyển chúng thành các quyết định thiết kế **khả thi hơn với sinh viên AI năm 3**. Mục tiêu không phải là thêm thật nhiều công nghệ, mà là biết **khi nào nên dùng**, **dùng ở mức nào** và **xếp chúng vào giai đoạn nào của roadmap**.

13.1 Điều chỉnh cho Bước 5 và Bước 6 – Parsing và Normalize

- Không nên xem parsing PDF học thuật là bài toán "rút text" đơn thuần. Với tài liệu nhiều bảng, sơ đồ và công thức, parser cần giữ được reading

order và loại block.

- Với bản đầu, nên ưu tiên parser có awareness về layout như **Docling** hoặc pipeline tương đương. Đây là lựa chọn thực dụng hơn so với việc đưa thẳng một mô hình Document AI nặng vào hệ thống.
- **LayoutLM** hoặc các mô hình vision-language cho document understanding là hướng rất hay nếu bạn muốn nghiên cứu sâu, nhưng thường phù hợp hơn với nhánh nghiên cứu Document AI hơn là một thành phần parse "cắm vào là chạy" cho đồ án MVP.
- Với tài liệu scan, OCR là bắt buộc; tuy nhiên OCR chỉ là một phần của bài toán, vì sau OCR bạn vẫn phải phục hồi lại thứ tự ngữ nghĩa của block.
- Normalizer phải theo nguyên tắc: **normalize nhẹ để retrieval tốt hơn, nhưng không được làm mất ký hiệu toán học, ký hiệu logic, tên biến, viết tắt và cấu trúc công thức**. Nếu cần, hãy giữ song song text gốc và text đã normalize.

Kết luận kỹ thuật cho đồ án

1. MVP: Docling hoặc parser hiểu layout + normalizer bảo toàn ký hiệu.
2. Nâng cấp: thêm OCR pipeline và parser chuyên sâu cho vùng công thức.
3. Nghiên cứu mở rộng: thử nghiệm LayoutLM hoặc mô hình vision-language cho parsing phức tạp.

13.2 Điều chỉnh cho Bước 11 và Bước 12 – Concept index, relation graph và hướng GraphRAG

- Góp ý xây "bản đồ kiến trúc môn học" là rất đúng hướng. Các câu hỏi kiểu prerequisite, lộ trình học, quan hệ giữa hai khái niệm gần nhau thường cần một lớp graph hoặc relation map.
- Tuy nhiên, cần phân biệt rõ: **Vector DB giải quyết retrieval**; còn **graph layer giải quyết quan hệ**. Metadata filter hoặc payload link trong vector DB không thay thế hoàn toàn graph traversal.
- Vì vậy, hướng khả thi là đi theo ba nấc:
  a. v1: chỉ dùng Qdrant + metadata tốt.
  b. v2: thêm concept index và relation graph lưu dưới dạng JSON/SQL hoặc graph nhẹ.
  c. v3: nếu cần nhiều câu hỏi prerequisite/path reasoning, thêm Neo4j hoặc graph store riêng.

- Đây chính là hướng GraphRAG ở mức thực dụng: không cần xây full graph ngay từ đầu, mà chỉ thêm lớp quan hệ đủ dùng cho câu hỏi học tập.

Kết luận kỹ thuật cho đồ án

1. Đừng bỏ qua concept index và relation graph.
2. Đừng nhầm metadata của vector DB với graph hoàn chỉnh.
3. Chỉ thêm Neo4j khi bạn thật sự cần traversal hoặc prerequisite reasoning rõ rệt.

13.3 Điều chỉnh cho Bước 15, Bước 16 và Bước 17 – Agentic AI và ReAct planner

- Ý tưởng dùng Query Decomposer như một agent là đúng, đặc biệt với câu hỏi đa bước.
- Nhưng với đồ án sinh viên, không nên bắt đầu bằng full agent tự do. Cách đó dễ tăng độ trễ, khó debug và khó đánh giá nguyên nhân lỗi.
- Hướng phù hợp hơn là **bounded agent** hoặc **planner kiểu ReAct có giới hạn**: chỉ có ít tool, ít vòng lặp và điều kiện dừng rõ ràng.
- Nên coi agent như **planner/controller** quyết định có cần thêm một vòng retrieval hay không, chứ không phải một thành phần được phép chạy vô hạn.

Quy tắc nên áp dụng nếu có agent

1. Giới hạn số vòng reasoning-retrieval.
2. Giới hạn số sub-query sinh ra.
3. Có tiêu chí dừng như “đã đủ evidence” hoặc “không cải thiện coverage”.
4. Mọi kết quả vẫn phải đi qua bước rerank và verification như pipeline chuẩn.

13.4 Điều chỉnh cho Bước 20 và Bước 23 – Verification, Self-RAG và Ragas

- Nhận xét rằng verification là bước khó nhất là hoàn toàn chính xác. Đây là nơi quyết định hệ thống có thật sự grounded hay không.

- Trong bản đầu, bạn nên tách rõ hai tầng:
  a. **Online verification**: chạy ngay trong runtime để kiểm tra claim, support, citation.
  b. **Offline evaluation**: dùng Ragas hoặc công cụ tương tự để đo Faithfulness, Context Precision/Recall và Answer Relevancy theo từng phiên bản hệ thống.
- **Self-RAG** là một định hướng rất tốt để nghiên cứu nâng cao, vì nó kết hợp adaptive retrieval và self-reflection. Tuy nhiên, đây nên là hướng nâng cấp sau MVP, không nên coi là yêu cầu bắt buộc của bản đầu.

Kết luận kỹ thuật cho đồ án

1. v1: verifier đơn giản nhưng chặt, kiểm tra support và citation.
2. v2: bổ sung self-reflection prompt hoặc critique pass.
3. v3: thử nghiệm chiến lược gần Self-RAG nếu nhóm còn thời gian và muốn đẩy sâu phần nghiên cứu.

13.5 Các thách thức đặc thù với dữ liệu Bách Khoa và tiếng Việt

- **Tiếng Việt**: mô hình embedding phải được benchmark thực tế trên corpus tài liệu của nhóm, không nên chọn chỉ vì thấy phổ biến. Dữ liệu học thuật tiếng Việt khác nhiều so với dữ liệu hội thoại hoặc tin tức.
- **Tài liệu chuyên ngành**: tài liệu kỹ thuật thường chứa rất nhiều ký hiệu đặc biệt, viết tắt, tên biến, công thức và sơ đồ. Nếu clean quá mạnh, retrieval sẽ giảm hit rate và verifier cũng khó đối chiếu lại claim với nguồn.
- **Tài liệu nhiều loại**: note, slide, handout, đề thi, lời giải và ảnh chụp màn hình có hình thức rất khác nhau. Vì vậy parser và chunker phải xét `source_type` ngay từ đầu.

Khuyến nghị chốt cho nhóm triển khai

1. Chạy được bản đơn giản trước: layout-aware parsing + chunk tốt + hybrid retrieval + verification.
2. Sau đó mới nâng dần: concept graph, bounded ReAct planner, verifier nâng cao.
3. Chỉ thêm GraphRAG sâu, agent mạnh hoặc Self-RAG khi đã có benchmark đủ tốt và đã kiểm soát được bản nền.

14 Phụ lục A – Data contracts khuyến nghị

14.1 QueryAnalysis

```
{
  "query_type": "comparison",
  "requires_multi_hop": true,
  "subject": "machine_learning",
  "preferred_content_types": ["definition", "example"],
  "difficulty": "medium"
}
```

14.2 EvidenceItem

```
{
  "chunk_id": "chunk_0020",
  "document_id": "doc_010",
  "role": "comparison_evidence",
  "score": 0.92,
  "page_start": 5,
  "page_end": 5,
  "text": "..."
}
```

14.3 FinalAnswer

```
{
  "answer": "...",
  "reasoning_summary": ["...", "..."],
  "citations": [
    {
      "document_id": "doc_010",
      "chunk_id": "chunk_0020",
      "page_start": 5,
      "page_end": 5
    }
  ],
  "confidence": 0.83
}
```

15 Phụ lục B – Skeleton codebase nên có

```
app/
  api/
    main.py
    routes_documents.py
    routes_chat.py
  core/
    config.py
    logging.py
  db/
    models.py
    repositories.py
  ingestion/
    storage.py
    parser_pdf.py
    cleaner.py
    chunker.py
    metadata.py
  retrieval/
    embedder.py
    sparse_index.py
    vector_store.py
    hybrid_retriever.py
    reranker.py
  knowledge/
    concept_extractor.py
    concept_index.py
    relation_graph.py
  reasoning/
    query_classifier.py
    query_decomposer.py
    planner.py
    evidence_builder.py
    synthesizer.py
  verification/
    verifier.py
    claim_checker.py
  evaluation/
    dataset_builder.py
    metrics.py
scripts/
  ingest_folder.py
  reindex_subject.py
  eval_rag.py
```

16 Kết luận

Nếu làm đúng theo tài liệu này, bạn sẽ đi theo một lộ trình rất rõ ràng:

1. Chuẩn hóa bài toán và data contract.
2. Xây knowledge base từ tài liệu học tập.
3. Tạo hybrid retrieval mạnh ở cả ngữ nghĩa và keyword.
4. Thêm lớp concept graph để hỗ trợ reasoning.
5. Dùng query decomposition, evidence set, reasoning synthesis và verification để trả lời các câu hỏi kiểu *why/how/comparison/problem-solving*.
6. Theo dõi chất lượng bằng evaluation và feedback loop.

Nói cách khác, bạn không chỉ xây một hệ thống "hỏi tài liệu", mà xây một **trợ lý tự học có suy luận dựa trên bằng chứng**. Đây mới là hướng phù hợp cho topic của bạn.